

Automated Network Reconnaissance and Vulnerability Assessment Tool

IT 359 Group Project — Final Project Writeup

Team Members: Dylan Voss & Grant Gollinger

Course: IT 359

Repository: github.com/dvoss26/IT-359-Group-Project

1. Introduction & Purpose (“The What”)

1.1 Project Summary

This project is an automated network reconnaissance and vulnerability assessment tool designed to assist in the early stages of a penetration test. The tool combines Bash scripting with industry-standard security utilities to automate the repetitive tasks that consume the majority of a tester's time during the reconnaissance phase: host discovery, port scanning, service enumeration, and basic risk identification. Once data is collected, the tool parses raw scan output and produces a structured, human-readable report that flags high-risk findings and offers preliminary mitigation guidance.

1.2 Problem Statement

Reconnaissance is the foundation of any penetration test, yet it is also the phase most prone to inconsistency and human error. Testers frequently retype the same Nmap commands, manually copy results between terminal sessions, and rewrite findings into client-ready notes. These manual steps slow assessments, introduce typographical errors, and produce reports that vary in quality from one engagement to the next. A small, disciplined automation layer can eliminate most of this friction without replacing the judgment of the tester.

1.3 Project Objectives

- Automate network reconnaissance tasks using custom Bash scripts.
- Demonstrate practical penetration testing concepts including host discovery, port scanning, and service enumeration.
- Analyze scan results programmatically to identify common security risks.
- Generate readable output that includes findings and mitigation recommendations.
- Adhere to ethical and legal standards for security testing throughout the project.
- Explore the use of locally hosted AI models to assist with scripting and output analysis.

1.4 Scope and Ethical Boundaries

The project is strictly educational. All scanning was conducted against systems owned by the project members or against lab environments where explicit authorization had been granted. No public, production, or unauthorized networks were targeted. The tool is intentionally limited to passive and active reconnaissance. It performs no exploitation, no credential attacks, and no actions that would modify a target system. This boundary is reinforced both in the project documentation and in the structure of the script itself, which exposes only enumeration commands.

2. Technical Implementation (“The How”)

2.1 Tools and Technologies

Component	Choice	Role in the Project
Scripting language	Bash	Orchestrates the scanning workflow and parses output.
Primary scanner	Nmap	Performs host discovery, port scanning, and service detection.
Operating system	Linux	Native environment for Nmap and standard text-processing tools.
Text utilities	grep, awk, sed	Extract and reformat fields from raw scan output.
AI assistance	Local models (e.g., gpt-oss, gemma)	Assists with script refinement and output review.
Version control	Git / GitHub	Tracks changes and supports collaboration between team members.

2.2 System Architecture

The tool is structured as a single Bash script that drives a five-stage pipeline. Each stage produces an intermediate artifact (a raw scan log, a parsed list, or a filtered finding) that is consumed by the next stage. Keeping the stages loosely coupled makes the script easier to debug because each stage can be inspected in isolation. The final stage assembles all intermediate artifacts into a single readable report.

2.3 The Reconnaissance Pipeline

Host Discovery. The script accepts a target IP address or CIDR subnet and uses Nmap to identify live hosts. This narrows the scope of subsequent scans to systems that are actually reachable, which both shortens runtime and reduces unnecessary network traffic.

Port Scanning. For each live host, the script performs a TCP port scan to determine which services are listening. Output is captured in a normal-format Nmap log so it can be parsed deterministically by later stages.

Service Enumeration. Service banners and version strings are gathered using Nmap's service detection capabilities. This information is what makes downstream risk analysis meaningful: “port 23 open” is far more useful when paired with “Telnet 0.17”.

Risk Identification. A set of pattern-matching rules implemented with grep, awk, and sed inspects the enumerated services and flags well-known concerns. Examples include unencrypted protocols such as FTP and Telnet, exposed database services, and ports historically associated with high-impact vulnerabilities.

Report Generation. All collected information is written to a timestamped output file. The report lists live hosts, open ports, detected services and versions, and the flagged risks together with short mitigation suggestions. The format is plain text so the report can be reviewed in any terminal or text editor.

2.4 Risk Identification Logic

The risk-flagging stage is intentionally simple and rule-based rather than relying on a vulnerability database. Each rule is a short conditional check that maps a service or port pattern to a category of concern. Telnet and FTP are flagged as unencrypted remote-access protocols. Open database ports such as MySQL or MSSQL are flagged because databases generally should not be reachable from outside their application tier. Legacy services, default web administration interfaces, and other common findings are matched in the same way. Because the rules are expressed in plain shell logic, new checks can be added in a single line without altering the rest of the pipeline.

2.5 AI-Assisted Development

Locally hosted language models were used as a development aid rather than as a runtime component. The team evaluated several open models to determine which were most useful at each stage of the project: one model assisted with refining and commenting Bash scripts, while another helped review scan output for clarity and structure. The models ran locally to ensure that no scan data ever left the project environment. The final tool itself does not depend on AI; the assistance was confined to the engineering process behind it.

2.6 Repository Layout and Documentation

The GitHub repository contains the project source, a detailed README describing the methodology, ethical considerations, and execution plan, and this writeup. Commit history was used to track the project's progression through environment setup, script development, scanning, parsing, reporting, and final validation.

3. Justification & Analysis (“The Why”)

3.1 Why Bash and Nmap

Bash and Nmap were chosen because they reflect the tools an entry-level penetration tester is most likely to encounter in the field. Nmap is the de facto standard for network reconnaissance, and Bash is the lingua franca of Linux operations. Building on these tools means the project produces transferable skills: the same techniques used in the script also work directly on a tester's command line. A more elaborate stack—Python with custom socket logic, for instance—would have added complexity without meaningfully improving the educational value of the work.

3.2 Why Automate the Reconnaissance Phase

Reconnaissance is the phase of a penetration test where automation pays the largest dividend. The work is highly procedural, the inputs are well defined, and the outputs follow consistent formats. By automating these steps, the tester is freed to focus on interpretation and follow-up rather than on command syntax. Automation also produces consistency: two runs of the script against the same target will perform exactly the same checks in exactly the same order, which is essential for repeatable assessments and after-action review.

3.3 Why Rule-Based Risk Identification

An intentional design choice was to use simple shell-level rules rather than integrating a full vulnerability scanner. A full scanner would have introduced licensing, dependency, and ethical considerations far beyond the scope of a course project. The rule-based approach keeps the script understandable end-to-end: a reader can trace exactly why a given finding was flagged. It also reflects how working testers actually operate, since experienced testers frequently rely on quick pattern matching against scan output to prioritize what to investigate next.

3.4 Strengths

- **Simplicity.** The pipeline is small, transparent, and easy to extend.
- **Repeatability.** The same input produces the same scan and the same report.
- **Low overhead.** The tool uses utilities that are already present on most Linux systems.
- **Clear ethical posture.** By design, the tool performs no exploitation or modification.
- **Educational value.** Each stage of the script maps directly to a real penetration testing concept.

3.5 Limitations

- **Surface-level analysis.** The rule set identifies common concerns but does not perform deep vulnerability assessment.
- **TCP focus.** UDP services and more advanced enumeration techniques are outside the current scope.
- **No authenticated scanning.** The tool only sees what is visible to an unauthenticated network observer.

- **Static rules.** New service patterns require manual updates to the script.

3.6 Alignment with Course Goals

The project ties directly to the course themes of secure systems, scripting, and responsible technology use. It demonstrates command of the Linux command line, structured automation with Bash, the basics of network security assessment, and the ethical framework that surrounds offensive security work. The supporting use of local AI tools also reflects the course's emphasis on adopting modern technologies responsibly: the AI assisted with engineering, but the design decisions, ethical boundaries, and final code review remained with the human team.

3.7 Future Improvements

- Extend the risk rule set to cover additional services and modern attack surface (for example, exposed Kubernetes or Docker APIs).
- Add UDP scanning for protocols such as DNS, SNMP, and NTP that are commonly overlooked.
- Output reports in additional formats such as Markdown or HTML to support easier sharing.
- Introduce a configuration file so testers can tune scan intensity and rule selection without editing the script.
- Integrate optional CVE lookups for detected service versions, while keeping the offline rule-based mode as the default.

4. Conclusion

This project set out to show that a small, well-structured automation script can capture a meaningful portion of the reconnaissance workflow that a penetration tester performs by hand. The resulting tool takes a target, discovers live hosts, enumerates open ports and services, applies a transparent set of risk rules, and produces a readable report. It is not a replacement for a full security assessment, and it was never intended to be. It is a focused demonstration of how scripting, standard security utilities, and a clear ethical framework can come together to produce something genuinely useful.

The development process emphasized simplicity over feature breadth. Every stage of the pipeline was kept small enough to read and reason about in one sitting, and every risk rule was kept simple enough that its logic could be defended in a code review. Local AI tools were used to accelerate the engineering process, but no aspect of the design or the ethical scope was delegated to them. The team chose what to build, why to build it, and where the boundaries should lie.

Most importantly, the project reinforced a lesson that applies well beyond this course. Effective security work is a combination of technical skill and judgment: knowing which commands to run is only half of the job, and knowing why they should or should not be run is the other half. By scoping the tool tightly, documenting its limits, and constraining it to authorized environments, the team practiced both halves at once. The finished tool is small, but the habits it required—clear scope, transparent logic, and respect for the systems being examined—are exactly the habits that make security professionals trustworthy.